

TASK 1: PYTHON TO C++ CONVERSION**ORIGINAL PYTHON CLASS FOR STUDENT:**

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name
        self.age = age
        self.grade = grade

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}")

# Test
s = Student("John Doe", 20, "A")
s.display()
```

TRANSLATED C++ CODE (STUDENT.CPP):

```

#include
<iostream>
#include <string>
using namespace
std;

class
student {
private:
string
name;    int
age;
string
grade;

public:
    student(string n, int a, string g) : name(n), age(a), grade(g) {}

    void display() {
        cout << "name: " << name << ", age: " << age << ", grade: " << grade
<< endl;
    }
};

int main() {
    student s("john doe", 20,
"a");    s.display();
return 0;
}

```

COMPILATION AND EXECUTION (TERMINAL OUTPUT):

```

$ g++ student.cpp -o student
$ ./student
name: john doe, age: 20, grade: A

```

PYTHON OUTPUT:

```

$ python3 student.py
name: john doe, age: 20, grade: A

```

BOTH OUTPUTS MATCH PERFECTLY. HANDLED CONSTRUCTOR INIT LIST, PRIVATE MEMBERS, AND STRING TYPES.

TASK 2: JAVA TO PYTHON FUNCTION CONVERSION

ORIGINAL JAVA METHOD:

```
PUBLIC CLASS PRIMECHECKER {    PUBLIC STATIC BOOLEAN ISPRIME(INT NUM) {        IF (NUM <= 1)
RETURN FALSE;        FOR (INT I = 2; I <= MATH.SQRT(NUM); I++) {            IF (NUM % I == 0) RETURN FALSE;
        }
        RETURN TRUE;
    }
}
```

TRANSLATED PYTHON FUNCTION (PRIME.PY):

```
IMPORT MATH

DEF IS_PRIME(NUM):    IF NUM <= 1:
RETURN FALSE    FOR I IN RANGE(2,
INT(MATH.SQRT(NUM)) + 1):
        IF NUM % I == 0:
RETURN FALSE
RETURN TRUE

# TESTS
TEST_NUMBERS = [2, 17, 15, 1, 29, 4] FOR N IN TEST_NUMBERS:
    PRINT(F"{N}: {IS_PRIME(N)}")
```

TERMINAL OUTPUT:

```
$ PYTHON3 PRIME.PY
```

```
2: TRUE
```

```
17: TRUE
```

```
15: FALSE
```

```
1: FALSE
```

```
29: TRUE
```

```
4: FALSE
```

VERIFIED WITH MULTIPLE INPUTS. USED RANGE AND MATH.SQRT FOR PYTHONIC STYLE.

TASK 3: PSEUDOCODE TO PYTHON IMPLEMENTATION

PSEUDOCODE FOR BUBBLE SORT:

```
ALGORITHM BUBBLESORT(A)
  FOR I FROM 0 TO LENGTH(A)-1
    FOR J FROM 0 TO LENGTH(A)-I-1
      IF A[J] > A[J+1]
        SWAP A[J] AND A[J+1]
      END IF
    END FOR
  END FOR
END ALGORITHM
```

TRANSLATED PYTHON (BUBBLE.PY):

```
DEF
BUBBLE_SORT(ARR):
N = LEN(ARR)  FOR
I IN RANGE(N):
    FOR J IN RANGE(0, N - I - 1):
        IF ARR[J] > ARR[J + 1]:
            ARR[J], ARR[J + 1] = ARR[J + 1], ARR[J]

# TEST NUMBERS = [64, 34, 25, 12,
22, 11, 90] PRINT("ORIGINAL:",
NUMBERS)
BUBBLE_SORT(NUMBERS)
PRINT("SORTED:", NUMBERS)
```

TERMINAL OUTPUT:

```
$ PYTHON3 BUBBLE.PY
ORIGINAL: [64, 34, 25, 12, 22, 11, 90]
SORTED: [11, 12, 22, 25, 34, 64, 90]
```

VALIDATED WITH SAMPLE INPUT. SWAPS IMPLEMENTED DIRECTLY

TASK 4: SQL TO PANDAS QUERY

SAMPLE SQL QUERY:

```
SELECT NAME, SALARY FROM EMPLOYEES WHERE SALARY > 50000;
```

SAMPLE DATAFRAME SETUP AND PANDAS CODE (PANDAS_QUERY.PY):

```
IMPORT PANDAS AS PD
```

```
DATA = {  
    'NAME': ['ALICE', 'BOB', 'CHARLIE', 'DIANA'],  
    'SALARY': [60000, 45000, 70000, 30000]  
}  
DF = PD.DATAFRAME(DATA)
```

```
# EQUIVALENT PANDAS QUERY RESULT = DF[DF['SALARY'] > 50000][['NAME', 'SALARY']] PRINT(RESULT)
```

TERMINAL OUTPUT:

```
$ PYTHON3  
PANDAS_QUERY.PY  
  
   NAME  
SALARY    0  
ALICE  60000  
2 CHARLIE  70000
```

FILTERS CORRECTLY MATCHING SQL. USED BOOLEAN INDEXING

TASK 5: REAL-TIME APPLICATION ALGORITHM TRANSLATION

SELECTED: PYTHON BINARY SEARCH TO C.

ORIGINAL PYTHON (BINARY_SEARCH.PY):

```

DEF BINARY_SEARCH(ARR, TARGET):
    LOW, HIGH = 0, LEN(ARR) - 1
WHILE LOW <= HIGH:
    MID = (LOW + HIGH) // 2
    IF ARR[MID] ==
TARGET:
        RETURN MID
    ELIF ARR[MID] <
TARGET:
        LOW = MID + 1
    ELSE:
        HIGH = MID - 1
RETURN -1

# TEST ARR =
[1, 3, 5, 7, 9]
PRINT(BINARY_SEARCH(ARR, 5)) # OUTPUT: 2

```

TRANSLATED C (BINARY_SEARCH.C):

```

#include <stdio.h>

INT BINARY_SEARCH(INT ARR[], INT N,
INT TARGET) {    INT LOW = 0, HIGH = N -
1;    WHILE (LOW <= HIGH) {        INT MID
= (LOW + HIGH) / 2;        IF (ARR[MID] ==
TARGET) RETURN MID;        ELSE IF
(ARR[MID] < TARGET) LOW = MID + 1;
    ELSE HIGH = MID - 1;
    }
    RETURN -1;
}

INT MAIN() {
    INT ARR[] = {1, 3, 5, 7, 9};    INT N = 5;
    PRINTF("%D\n", BINARY_SEARCH(ARR, N, 5)); //
    OUTPUT: 2    RETURN 0;
}

```

TERMINAL OUTPUT FOR C:

```
$ gcc BINARY_SEARCH.C -o BINARY
```

```
$ ./BINARY
```

```
2
```

CHALLENGES FACED:

- C NEEDS ARRAY SIZE EXPLICITLY; PYTHON USES LEN().
- INTEGER DIVISION SAME, BUT MANUAL ARRAY HANDLING IN C.
- NO BUILT-IN LISTS; USED FIXED ARRAY.

BOTH VERSIONS RETURN SAME INDEX FOR TESTS.